



AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Summer 2025-2026

Section: D

Group No: 01

Project Report On

SkillHub – A freelance Marketplace Management System

Supervised By

Dr. Md. Iftekharul Mobin

Submitted By:

Name			ID		
1. MD. Nazmus Sakib			24-58148-2		
2. Md. Omar Faruk Aumi			24-59497-3		
3. Sadman Ahmed			25-62451-2		
4. Anika Sumaiya			24-59063-3		
Obtained Marks for CO2 and CO3 (Description given in the following page)					
Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria (CO2)	Total =		Evaluation Criteria (CO3)	Total =	
Requirement fulfillment			Organization of the application		
Validation			Representation and Integration of Database		
Verification			Graphical User Interface		

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				
Requirement fulfillment	Fails to demonstrate any understanding of real-life scenario-based project development or functional requirement identification. There is no attempt to depict a project or identify functional requirements accurately.	Demonstrates limited understanding of real-life scenario-based project development and functional requirement identification. The project depicted lacks coherence or relevance to real-life scenarios, and functional requirements are inaccurately identified or insufficiently described.	Presents a basic depiction of a real-life scenario-based project and identifies some functional requirements. However, the project lacks depth or complexity, and some functional requirements may be vaguely defined or missing key details.	Effectively demonstrates a realistic scenario-based project and accurately identifies most functional requirements. The project is well-developed with appropriate complexity, and functional requirements are clearly articulated with relevant details.	Exhibits an exceptional understanding of real-life scenario-based project development and accurately identifies all functional requirements. The project is meticulously developed with thorough attention to detail, reflecting a comprehensive understanding of Object-Oriented Programming project development activities.
Validation	Fails to demonstrate any understanding or implementation of validation forms in their system. There is no attempt to deal with data validation, and validation requirements are completely ignored or incorrectly applied.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation.	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation.	Effectively demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system.	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall reliability and integrity of the system.
Verification	Fails to demonstrate any attempt to verify the system data or functional requirements. There is no evidence of understanding or implementation	Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts are incomplete or	Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may be inconsistent or	Identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and thorough, with attention to	Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive

	of verification processes, and data flow is not considered.	inaccurate, and there is insufficient consideration given to ensuring data integrity and functionality.	lack thoroughness, and there may be gaps in ensuring proper functional requirements and data flow.	ensuring data integrity and appropriate data flow within the system.	and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.
--	---	---	--	--	--

CO3: Prepare and Explain a real life desktop based application synthesizing several component of C# along with development tools to adhere the given requirements.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				
Organization of the application	Fails to identify any suitable real time application or requirements for project development activities related to OOP.	Limited understanding about the project scopes and scenarios or identification of functional requirements.	Lacks depth or relevance to OOP project development activities and may contain inaccuracies. Real-life scenarios are mentioned, but the discussion lacks depth or clarity.	Consider and integrate the idea of several core aspects of the project along with relevance to real-life scenarios. Demonstrating a solid understanding of the application presentation.	Generalize and exhibits an exceptional understanding of project preparation according to a to real-life scenarios. Also contains proper and insightful identification of the system which is comprehensive and precise.
Representation and Integration of Database	Fails to identify and present any understanding or implementation of database. Also failed to integrate the data with the project itself.	Limited understanding of the database concepts or their proper way of using in a real time project. While some attempt may be made to implement but it is incomplete or poorly executed, leading to inadequate design.	Lacks depth or relevance to database integration with the application. Shows a basic understanding but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistency. May lack proper normalization.	Integrate the database with the forms properly and implements it with proper validation which is mostly accurate and comprehensive, ensuring the proper handling of data input and verification along with general normalization.	Exhibits an exceptional understanding and implementation of database ensuring attention to detail, and robust data manipulation procedures and contributing to the overall clarity.
Graphical User Interface	Fails to present or prepare GUI based application interfaces. There is no evidence of creating or integrating such things according to their usefulness.	Limited understanding of graphical user interfaces. Lack of design knowledge. Very poor attempt to make such things which are currently obsolete or can't be identified as coherent.	Shows a basic understanding of creating user interfaces. Most of them are interconnected but maybe some of them lack it. However, most of it can be described as user friendly.	Effectively identifies and meet the consider the simplicity. Design related works are mostly accurate and taken proper attention to ensuring a user-friendly coherent system.	Exhibits an exceptional work design following a high standard of simple and elegant work. Several controls and mechanism has been organized in a preferred way according to the coherent usage .

Table of Contents

1. Project Title.....	Error! Bookmark not defined.
2. Introduction	Error! Bookmark not defined.
3. Case Study.....	Error! Bookmark not defined.
4. Functionalities	Error! Bookmark not defined.
5. ER Diagram, Normalization and Database Schema.....	Error! Bookmark not defined.
6. SQL Queries and Database Integration	Error! Bookmark not defined.
7. Screen Navigation and Implemented Form Designs.....	Error! Bookmark not defined.
8. Conclusion	Error! Bookmark not defined.

1. Project Title

SkillHub - Freelance Marketplace Management System

2. Introduction

SkillHub is a completed C# Windows Forms desktop application that connects clients with software-service freelancers through one managed marketplace. The final system supports account registration and secure login, service discovery, cart and checkout, order progress, delivery approval, reviews, disputes, platform commission, freelancer earnings and administrative control.

The application uses C# and Windows Forms for the graphical interface, SQL Server for persistent storage, and ADO.NET for database access. The final solution follows a layered structure: forms handle interaction, services apply business rules, repositories execute parameterized SQL, shared models carry data, and a single database connection class reads the configured connection string.

The academic role names are retained for compatibility with the project brief: SUPER_ADMIN represents the platform owner, ADMIN represents a freelancer, and CUSTOMER represents a client. The final database also stores readable role names - Admin, Freelancer and Client - through the Roles table.

Project Objectives

- Build one integrated desktop application for three user roles.
- Apply object-oriented design through focused classes, encapsulated responsibilities and reusable services.
- Maintain normalized and verifiable data in SQL Server.
- Protect credentials, validate user input and isolate records by the signed-in user's database ID.
- Complete the full marketplace flow from service discovery to payment, delivery approval and freelancer earnings.

Development Responsibilities

Team Member	Primary Responsibility
MD. Nazmus Sakib	Database foundation, authentication, account/profile CRUD, project integration, conflict resolution and final verification
Md. Omar Faruk Aumi	Super Admin and financial management module
Sadman Ahmed	Freelancer profile, service and order-management module
Anika Sumaiya	Client browsing, cart, checkout and order-tracking module

3. Case Study

Nabila runs a small business and needs an inventory management application. Instead of contacting several developers separately, she signs in to SkillHub as a client, searches the available services, filters the results by category and availability, and opens the details of a suitable C# and database service.

Rafi is a registered freelancer. His profile contains a professional title, biography, skills, profile image, rating and verification information. He publishes services with a category, description, price, delivery time, image and available capacity. Nabila reviews this information, adds the service to her cart and completes the simulated checkout.

The checkout operation creates the order and payment records, reduces the available capacity and clears the cart as one controlled transaction. Rafi sees only the orders assigned to his user ID. He accepts the work, changes the status to In Progress and then submits the delivery.

Nabila sees the delivered order in her order history and approves it after checking the work. The order becomes Completed, SkillHub keeps the configured 10 percent platform commission, and one credit is added to Rafi's wallet. A duplicate wallet credit is blocked for the same completed order.

If the service or delivery is unacceptable, Nabila can open a dispute. The Super Admin can inspect the complaint, payment and order history, record a decision, manage account status and review platform reports. This case study represents the complete implemented flow shown later in the report.

4. Functionalities

Common and Security Functions

- Register as a client or freelancer and sign in with an email and password.
- Store passwords as PBKDF2-SHA256 hashes with 120,000 iterations; plain-text passwords are not stored or compared.
- Check account status and role before routing the user to the correct dashboard.
- Start one UserSession containing the signed-in UserId, name and role; clear it during logout.
- View and update profile information, change a password and receive clear validation messages.

SUPER ADMIN - Platform Owner

- View platform totals and quick links for accounts, categories, offers, payments, disputes, withdrawals, settings and reports.
- Create, search, update, deactivate and reactivate user accounts while blocking unsafe self-deactivation.
- Manage categories, offers, reviews, disputes and withdrawal decisions.
- Inspect revenue, platform commission, freelancer earnings and freelancer-performance reports.

ADMIN - Freelancer / Service Provider

- Maintain a professional profile with biography, skills, profile image, rating and verification status.
- Create, read, update and deactivate services with artwork, category, price, delivery days and available slots.
- See only the services and orders owned by the signed-in freelancer.
- Accept orders, mark work In Progress and submit delivery.
- Monitor low capacity, sales, commission deductions, wallet credits and withdrawal requests.

CUSTOMER - Client

- Browse visual service cards and search by title, description, freelancer, category or skill.
- Filter and sort services by category, availability, rating, price and delivery information.
- Open complete service and freelancer details before purchasing.
- Add services to the cart, change quantity, remove items and complete a simulated checkout.
- Track order status, approve delivery, submit a verified review or open a dispute when eligible.

Core Business Rules

Rule	Implemented Control
Account safety	Required fields, unique email, password confirmation, role checks and soft deactivation
Data isolation	Repositories use UserSession.UserId instead of hard-coded account IDs
Service capacity	Available slots cannot fall below zero; unavailable services cannot be added for a new purchase
Checkout consistency	Order, payment, capacity and cart changes commit or roll back together
Earnings integrity	One wallet credit is allowed for each completed paid order
History preservation	Deactivation keeps linked orders, payments, reviews and financial records

5. ER Diagram, Normalization and Database Schema

The final SkillHubDB database contains 16 application tables. The SSMS evidence below is taken from the implemented database and is used as the authoritative physical table list for this report.

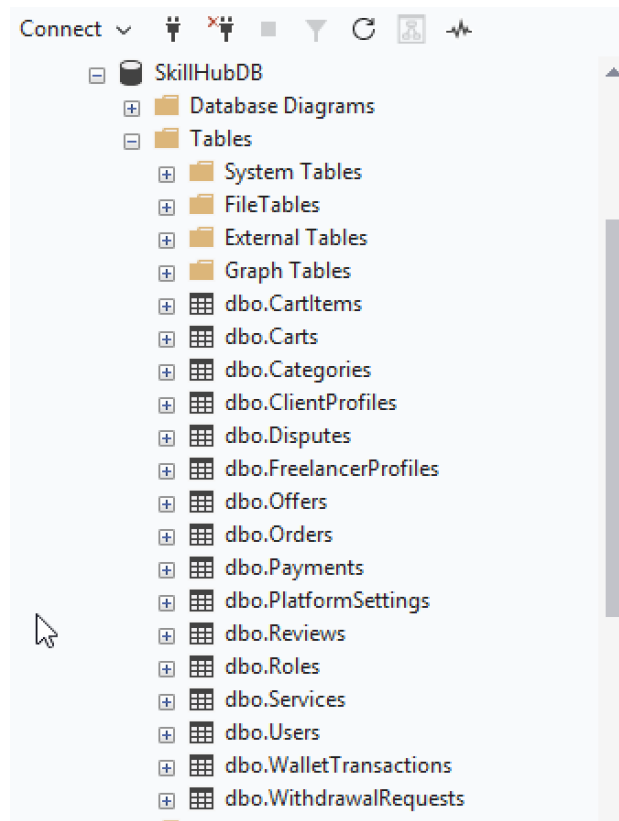


Figure 1. Implemented SkillHubDB table list in SQL Server Management Studio.

Entity Relationship Model

The logical ER model separates users, freelancer information, services, orders, payments, reviews, disputes and wallet events. The final implementation expands the identity area with Roles and ClientProfiles. CartItems remains the repeating cart detail structure, while the final order workflow records the service relationship used by the application.

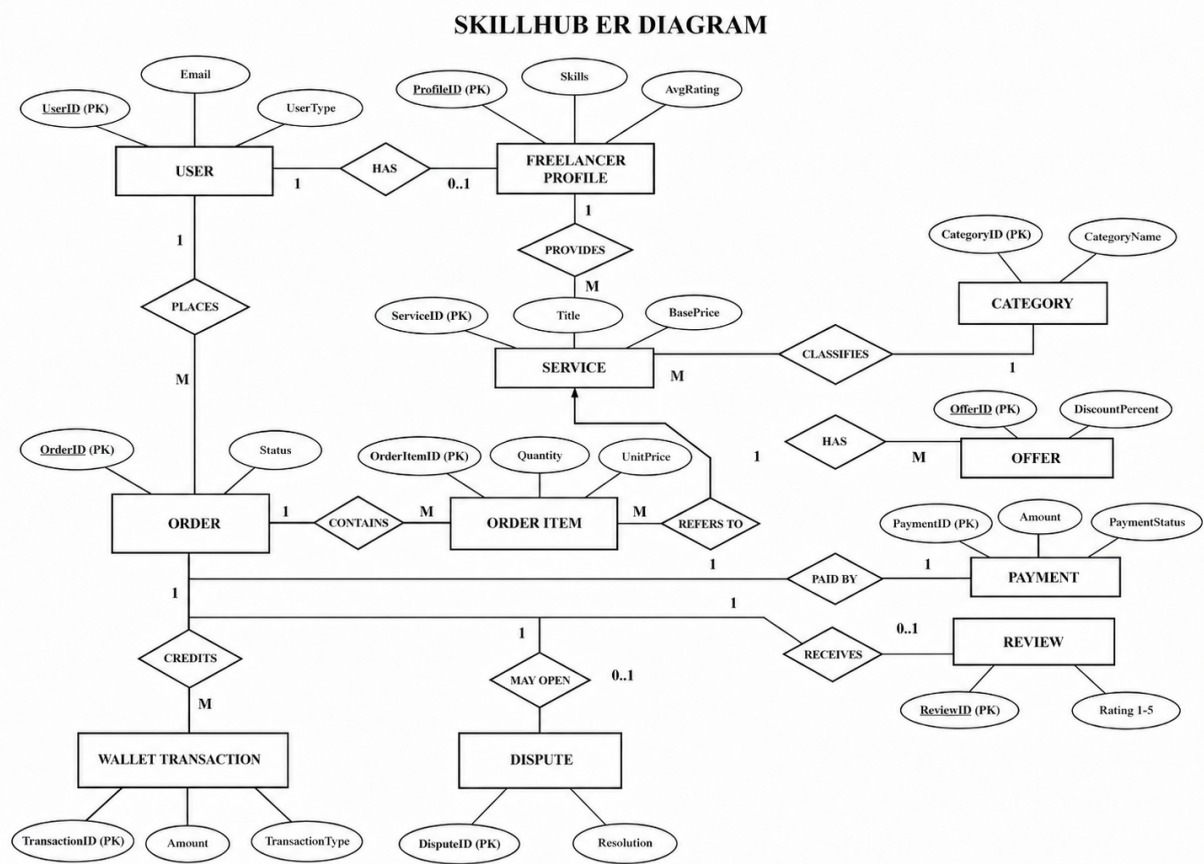


Figure 2. Logical SkillHub ER model for the core marketplace transaction.

Normalization and Finalization

First Normal Form (1NF)

Each table contains atomic values and one record per business object. Repeating services in a cart are stored as separate CartItems rows, and each payment, review, dispute, wallet entry and withdrawal request has its own record.

Second Normal Form (2NF)

Non-key attributes depend on the complete key of their table. Profile fields are separated from the common Users record, service information is stored once in Services, and cart detail values depend on the selected cart and service.

Third Normal Form (3NF)

Transitive data is separated into related tables. Role descriptions are held in Roles, category names are held in Categories, freelancer and client details are held in their profile tables, and transaction events reference the relevant user, service or order through foreign keys. This avoids repeating descriptive values and keeps updates consistent.

Implemented Data Dictionary

Table	Primary Key	Main Relationship	Purpose
Roles	RoleId	Users	Readable role definitions and mapping for Admin, Freelancer and Client
Users	UserId	Roles	Authentication, identity, role, status and profile-image path
ClientProfiles	ClientId	Users	Client-specific profile information
FreelancerProfiles	FreelancerId	Users	Professional title, bio, skills, rating and verification
Categories	CategoryId	Services	Master service categories and active status

Table	Primary Key	Main Relationship	Purpose
Services	ServiceId	Users; Categories	Freelancer listing, artwork, price, delivery time and capacity
Offers	OfferId	Services	Discount offer and validity information
Carts	CartId	Users	One active shopping cart for a client
CartItems	CartItemId	Carts; Services	Selected service, quantity and price snapshot
Orders	OrderId	Users; Services	Client-freelancer transaction, status and financial totals
Payments	PaymentId	Orders	Simulated payment method, reference, amount and state
Reviews	ReviewId	Orders; Users	Verified rating, comment and moderation state
Disputes	DisputeId	Orders; Users	Complaint, status, resolution and responsible administrator
WalletTransactions	TransactionId	Users; Orders	Freelancer credits, withdrawals, refunds and adjustments
WithdrawalRequests	WithdrawalId	Users	Requested amount, decision, processor and processing time
PlatformSettings	SettingKey	Users	Commission percentage and configurable platform values

Database Verification

The read-only verification script checks the physical tables, roles, seeded accounts, categories, services and platform settings. The executed result shows 16 tables, the three required roles, active sample accounts and a default commission value of 10.00 percent.

SQL Query 1.s...ackup (67)*

Object Explorer Details

100 %

22

0

Ln: 204, Ch: 1

SPC

LF

UTF-8

Results

Messages

/	/	Freelancer	ADMIN	Farhan Kabir	farhan@skillhub.local	Assets\Profiles\Farhan-kabir....	Active
8	8	Freelancer	ADMIN	Samira Islam	samira@skillhub.local	Assets\Profiles\samira-islam....	Active

	CategoryCode	CategoryName	IsActive
3	DES-02	UI/UX Design	1
4	DEV-01	Full-Stack De...	1
5	DEV-02	Frontend Dev...	1
6	DEV-03	Backend De...	1
7	EMB-01	Rust and Fim...	1
8	MED-01	Software-Rel...	1
9	MED-02	Software-Rel...	1
10	MOB-01	Android Appli...	1

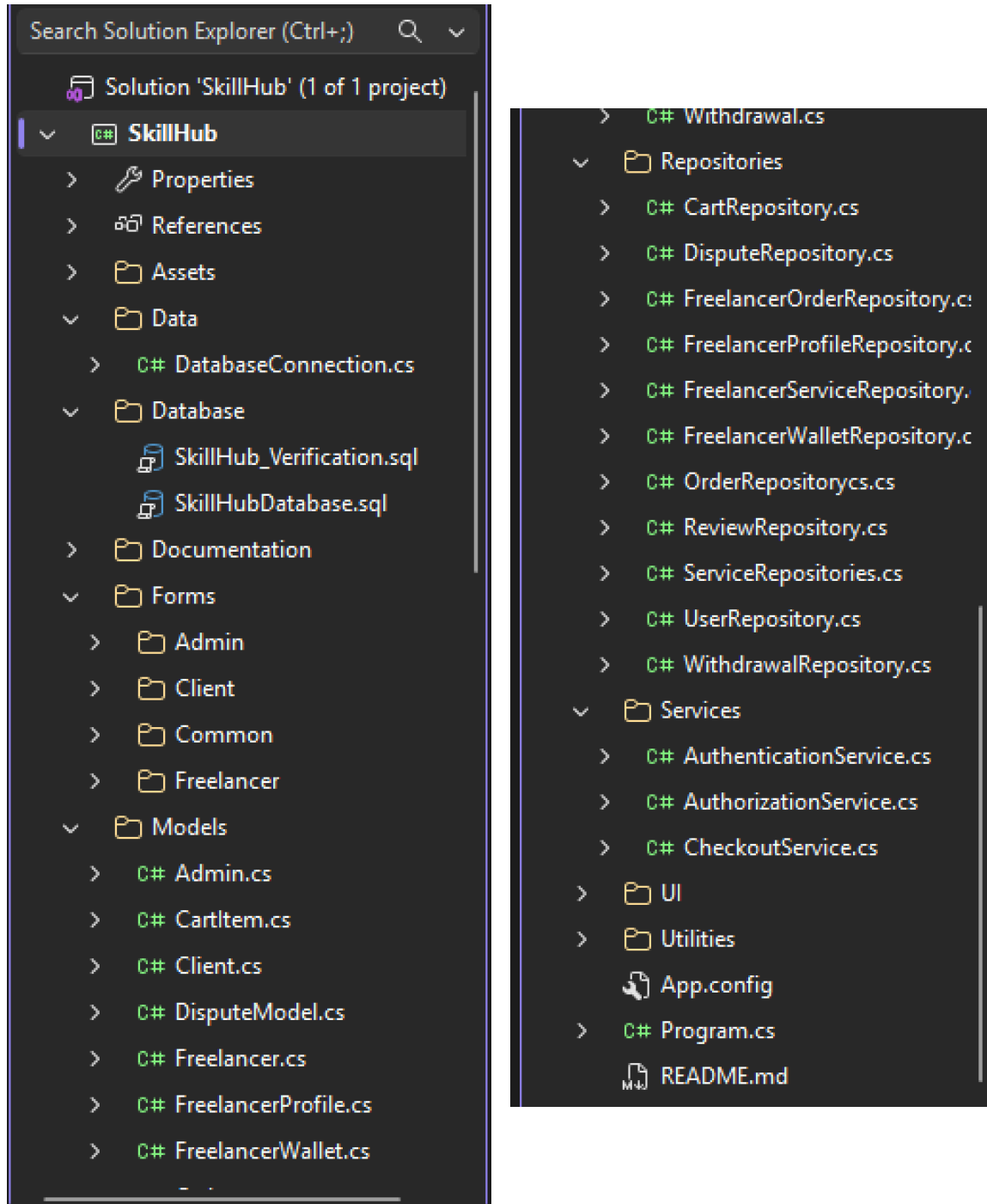
	ServiceId	FreelancerName	CategoryName	Title	ServiceImagePath	Price
3	3	Rafi Ahmed	Rust and Firmware De...	Develop ESP32 Firmware and Embedde...	Assets\Services\embedded-iot.png	6000.00
4	4	Ayesha Karim	Frontend Development	Build a Responsive React Business Das...	Assets\Services\frontend-web.png	4200.00
5	5	Ayesha Karim	UI/UX Design	Design a Complete Mobile App UI/UX Pr...	Assets\Services\uiux-design.png	3000.00
6	6	Tanvir Hasan	Backend Development	Develop a Secure ASP.NET Core REST...	Assets\Services\backend-api.png	5500.00
7	7	Tanvir Hasan	Backend Development	Optimize a SQL Server Database for Perf...	Assets\Services\backend-api.png	3200.00
8	8	Nusrat Jahan	Data Analysis	Build an Interactive Power BI Analytics D...	Assets\Services\data-analytics.png	2800.00
9	9	Nusrat Jahan	Quality Assurance	Create a Complete QA Test Plan and Bu...	Assets\Services\quality-assurance.png	2200.00
10	10	Farhan Kabir	Cybersecurity	Perform an Authorized Web Application ...	Assets\Services\cybersecurity.png	6500.00

	SettingKey	SettingValue	Description
1	CommissionPercent	10.00	Default platform commission percentage for new o...

Figure 3. Executed SQL verification results and seeded SkillHub data.

6. SQL Queries and Database Integration

Database access is centralized so forms do not build connection strings or repeat low-level ADO.NET setup. DatabaseConnection creates and opens SqlConnection objects, while repository methods execute parameterized commands. Service classes validate input and apply business rules before a repository changes the database.



(a) Shared project folders

(b) Forms grouped by role

Figure 4. Integrated solution structure separating data, services, repositories, models, utilities and forms.

```

12 public sealed class DatabaseConnection
13 {
14     1 reference
15     public const string ConnectionStringName = "SkillHubConnection";
16
17     2 references
18     private readonly string _connectionString;
19
20     36 references
21     public DatabaseConnection()
22     {
23         : this(ReadConfiguredConnectionString())
24     }
25
26     1 reference
27     public DatabaseConnection(string connectionString)
28     {
29         if (string.IsNullOrEmpty(connectionString))
30         {
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49

```

(a) Connection creation

```

27 throw new ConfigurationErrorsException(
28     "The SkillHub database connection string is missing.");
29 }
30
31 _connectionString = connectionString;
32
33
34
35 16 references
36 public SqlConnection CreateConnection()
37 {
38     return new SqlConnection(_connectionString);
39 }
40
41 59 references
42 public SqlConnection OpenConnection()
43 {
44     SqlConnection connection = CreateConnection();
45
46     try
47     {
48         connection.Open();
49         return connection;
50     }
51     catch
52     {
53
54
55
56
57
58
59

```

(b) Safe connection opening

Figure 5. Central database connection code reading the configured SkillHub connection string.

Authentication Lookup

The login query retrieves the stored hash and role information for one email. Password verification is then performed by PasswordHasher in C#, not by comparing a plain-text password inside SQL.

```

SELECT u.UserId, u.FullName, u.Email, u.PasswordHash,
       r.RoleName, u.UserType, u.Status
FROM Users AS u
INNER JOIN Roles AS r ON r.RoleId = u.RoleId
WHERE u.Email = @Email;

```

Service Search and Availability

Client filters are translated into parameters. The query joins Services with Users and Categories so the interface can show the service, freelancer and category together without duplicated storage.

```

SELECT s.ServiceId, s.Title, s.Description, s.Price,
       s.DeliveryDays, s.AvailableSlots,
       u.FullName AS FreelancerName, c.CategoryName
FROM Services AS s
INNER JOIN Users AS u ON u.UserId = s.FreelancerId
INNER JOIN Categories AS c ON c.CategoryId = s.CategoryId
WHERE s.IsActive = 1
      AND (@Keyword = N'' OR s.Title LIKE N'%'+ @Keyword + N'%'+
          OR s.Description LIKE N'%'+ @Keyword + N'%'+
          OR u.FullName LIKE N'%'+ @Keyword + N'%')
      AND (@CategoryId IS NULL OR s.CategoryId = @CategoryId)
      AND (@AvailableOnly = 0 OR s.AvailableSlots > 0);

```

Account Update and Soft Deactivation

Account records are updated by key. Deactivation changes status instead of deleting the row, which preserves linked marketplace and financial history.

```

UPDATE Users
SET FullName = @FullName,
    Email = @Email,
    Phone = @Phone,
    Address = @Address,
    Status = @Status
WHERE UserId = @UserId;

UPDATE Users
SET Status = N'Deactivated'
WHERE UserId = @UserId AND UserId <> @CurrentUserId;

```

Checkout Transaction

Checkout is handled as one transaction. If any required statement fails, the catch block rolls the transaction back so the order, payment, capacity and cart do not become inconsistent.

```
BEGIN TRY
    BEGIN TRANSACTION;
    INSERT INTO Orders (...) VALUES (...);
    INSERT INTO Payments (...) VALUES (...);
    UPDATE Services
        SET AvailableSlots = AvailableSlots - @Quantity
        WHERE ServiceId = @ServiceId AND AvailableSlots >= @Quantity;
    DELETE FROM CartItems WHERE CartId = @CartId;
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
    THROW;
END CATCH;
```

Completion and Idempotent Wallet Credit

When the client approves a delivered order, the order is completed and the freelancer is credited once. The OrderId check prevents accidental duplicate credits if the approval action is triggered again.

```
IF NOT EXISTS (
    SELECT 1 FROM WalletTransactions
    WHERE OrderId = @OrderId AND TransactionType = N'Credit'
)
BEGIN
    INSERT INTO WalletTransactions
        (FreelancerId, OrderId, TransactionType, Amount, CreatedAt)
    VALUES
        (@FreelancerId, @OrderId, N'Credit', @FreelancerAmount, SYSDATETIME());
END;
```

Credential and Session Code

The implementation validates required login values, checks the stored PBKDF2-SHA256 hash, rejects inactive accounts and starts UserSession only after successful authentication.

```
32 public User Login(string email, string password)
33 {
34     string normalizedEmail = InputValidator.RequireEmail(email);
35     if (string.IsNullOrEmpty(password))
36     {
37         throw new ArgumentException("Password is required.");
38     }
39     User user = _users.GetByEmail(normalizedEmail);
40     if (user == null || !PasswordHasher.VerifyPassword(password, user.PasswordHash))
41     {
42         throw new UnauthorizedAccessException("The email or password is incorrect.");
43     }
44     if (!user.IsActive)
45     {
46         throw new UnauthorizedAccessException("This account is not active. Contact the platform administrator.");
47     }
48     _users.RecordSuccessfulLogin(user.UserId);
49 }
```

(a) Authentication service

```
47 if (!user.IsActive)
48 {
49     throw new UnauthorizedAccessException(
50         "This account is not active. Contact the platform administrator.");
51 }
52 _users.RecordSuccessfulLogin(user.UserId);
53 UserSession.Start(user);
54 return user;
55 }
56 public int Register(
57     string roleName
58 )
59 {
60     ...
61 }
```

(b) Password hashing and verification

Figure 6. Authentication, account-status validation and protected password handling.

7. Screen Navigation and Implemented Form Designs

The following screenshots are taken from the final integrated application. They replace the earlier wireframes and show the working forms, data and validation states used during the demonstration.

Navigation and Role Routing

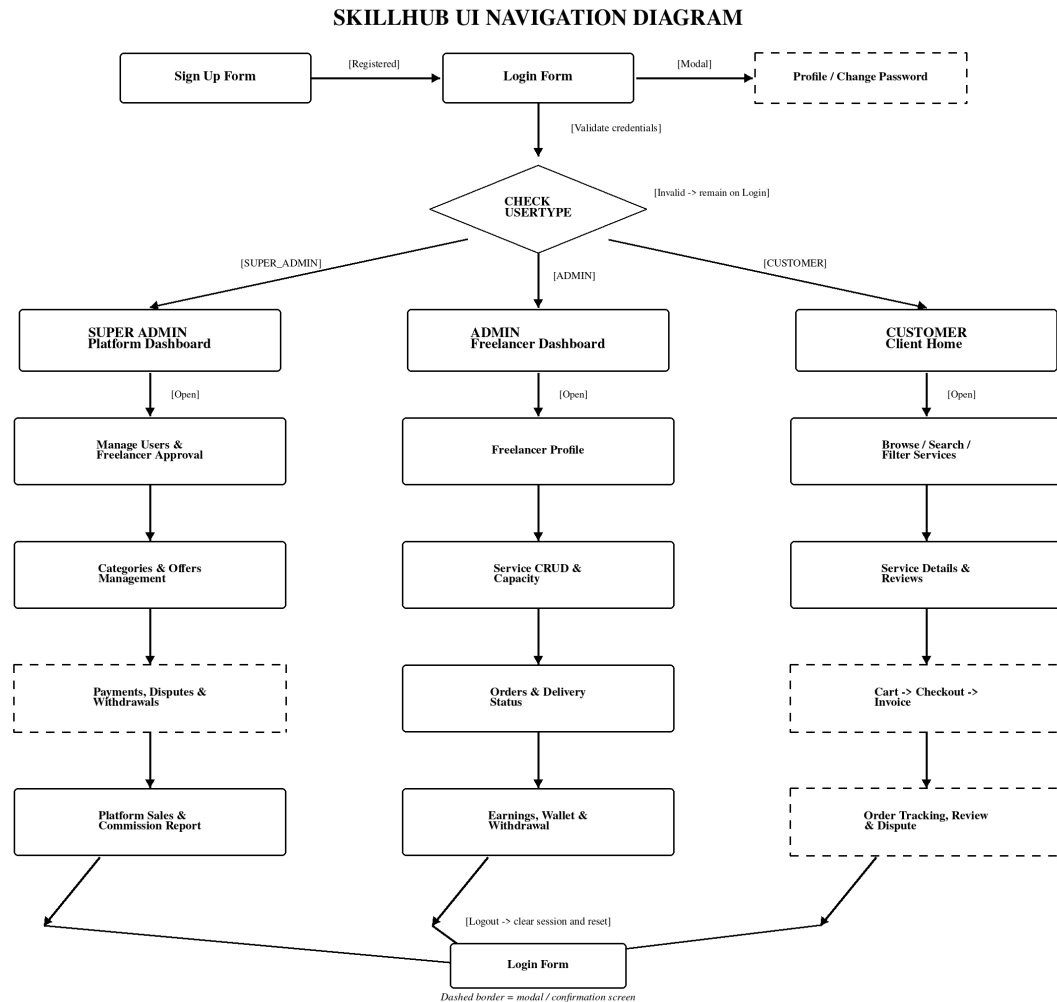


Figure 7. SkillHub navigation flow from sign-up and login to the three role dashboards.

Login and Role Dashboards

The login form accepts a registered email and password, keeps the password hidden and displays a readable error when credentials are invalid. After authentication, the stored role determines which dashboard opens.

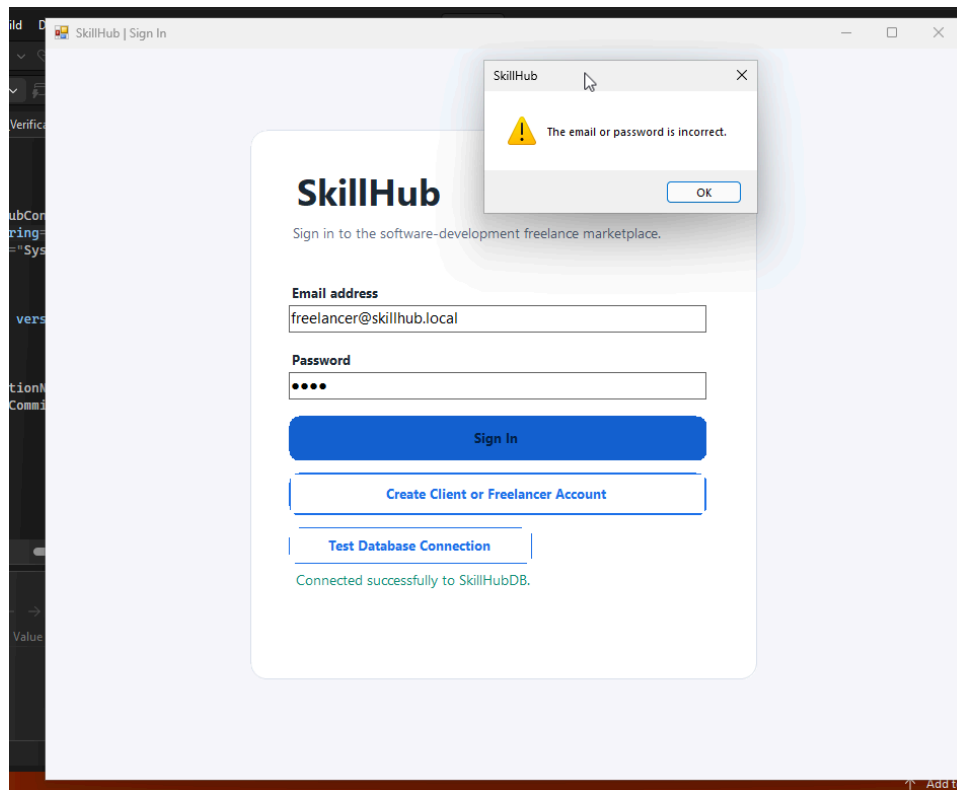


Figure 8. Implemented login form with credential validation.

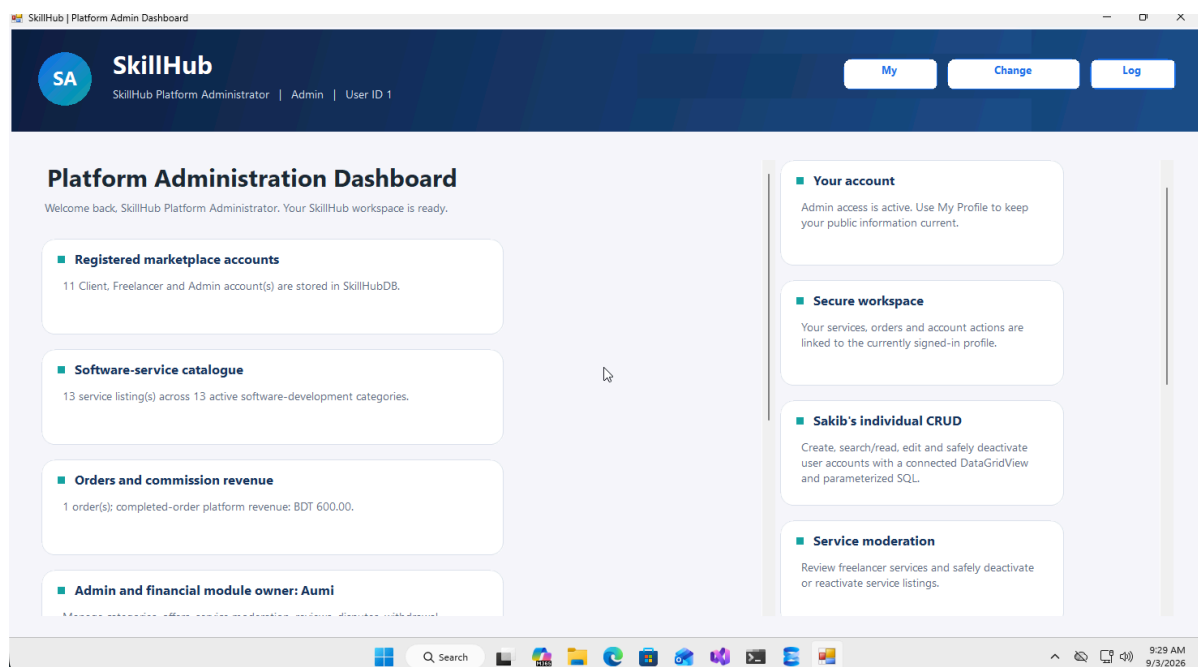


Figure 9. Super Admin dashboard with account, marketplace and reporting controls.

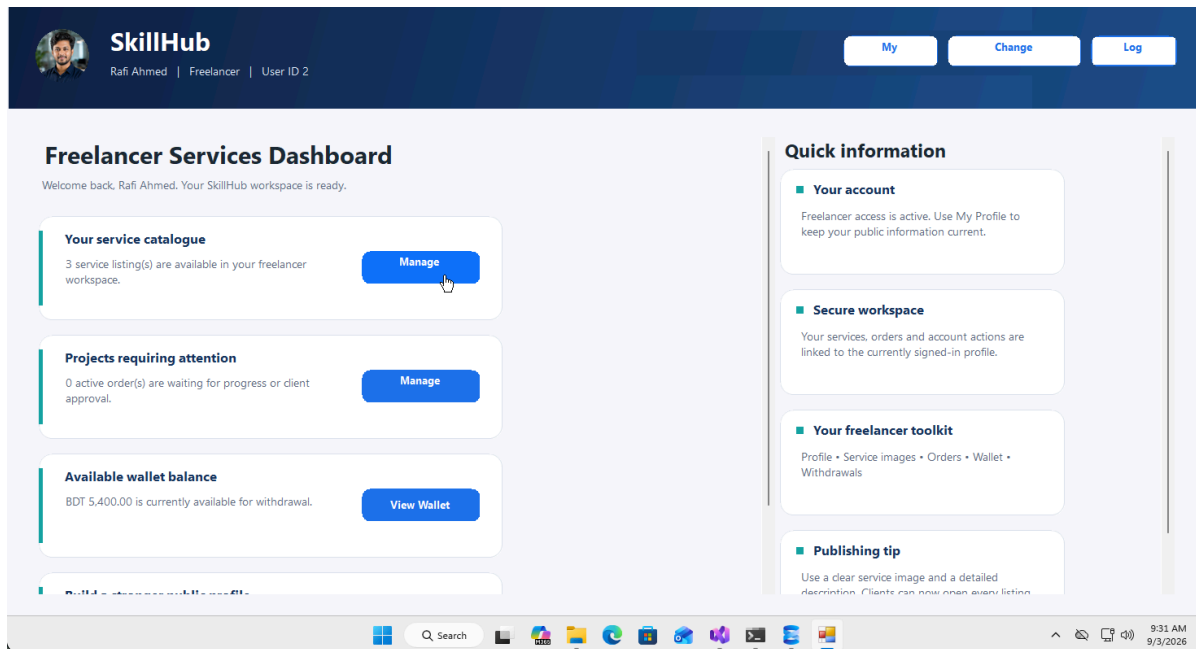


Figure 10. Freelancer dashboard with profile, service, order and earnings controls.

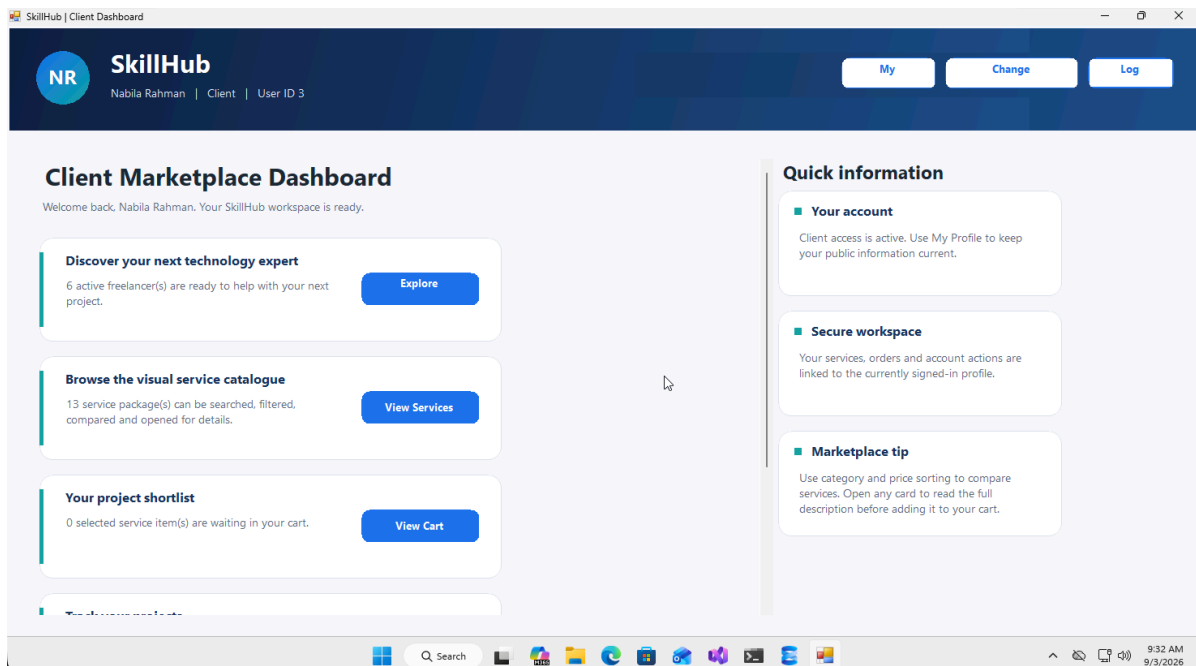


Figure 11. Client dashboard with marketplace, cart, order and profile controls.

Account and Profile CRUD

FrmAccountManager demonstrates the complete account lifecycle. The Super Admin can create an account, load and search records, select a row for editing, update the values and deactivate the account without deleting its history.

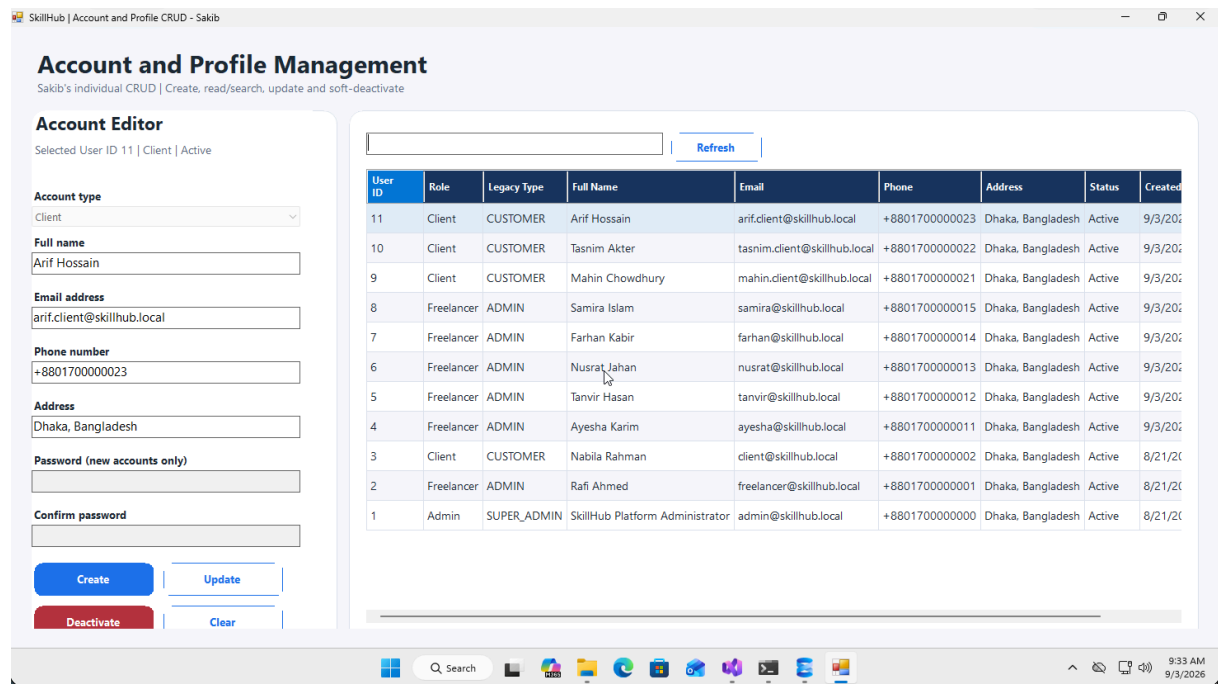
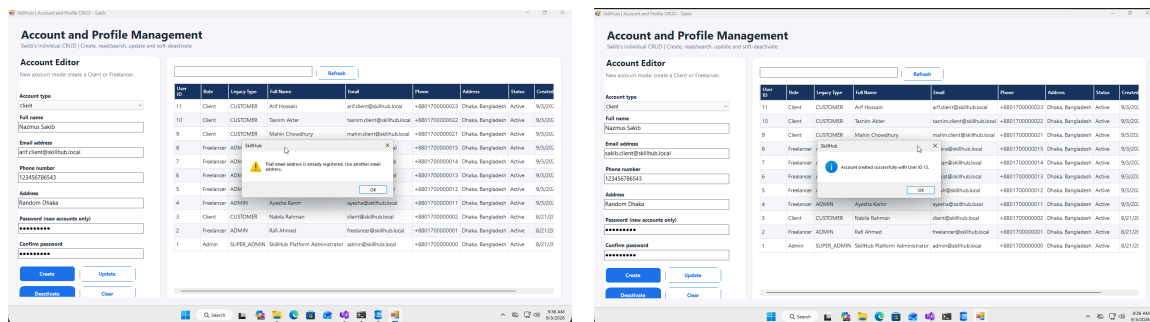


Figure 12. Account and Profile Management showing the editor, live search, records and CRUD controls.

Validation is visible at the interface and repeated in the service and database layers. Duplicate email, invalid password confirmation, missing required fields and unsafe self-deactivation are blocked before a valid database update is accepted.



(a) Duplicate-email warning

(b) Successful account update

Figure 13. Account CRUD validation and the verified updated record.

Client Module

The marketplace uses visual service cards instead of a plain table. The client can combine keyword search, category selection, availability filtering and sorting while still seeing the freelancer, rating, price, delivery time, image and remaining capacity.

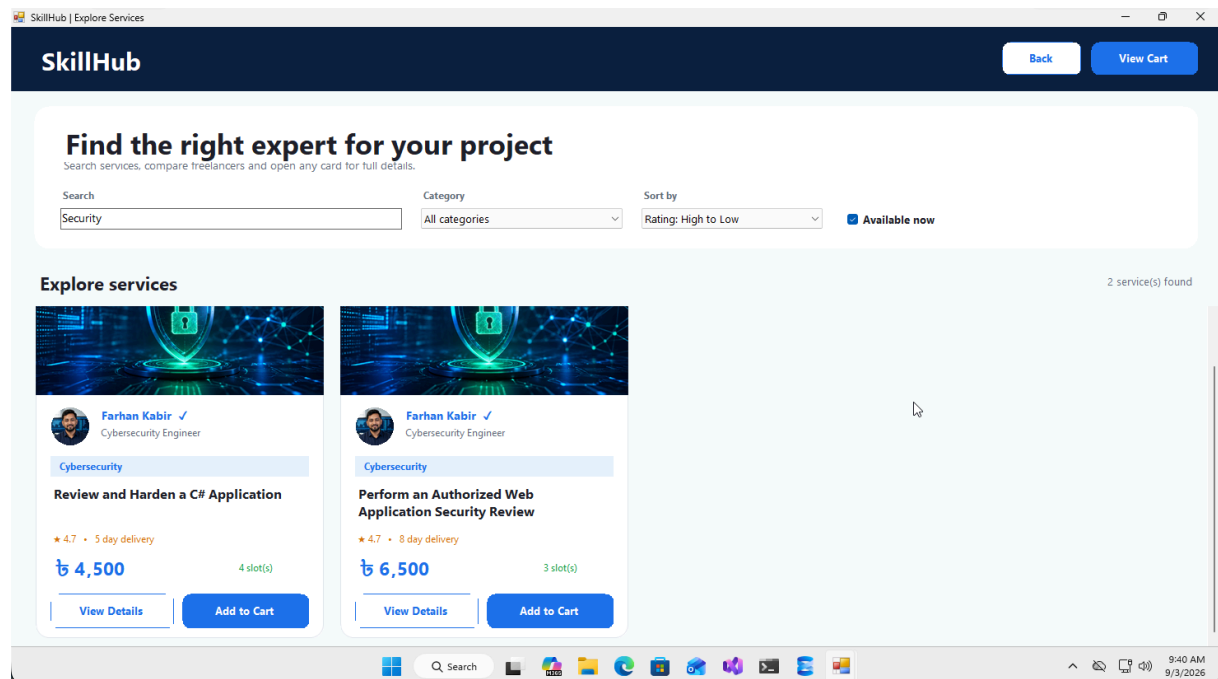


Figure 14. Client Explore Services screen with search, category, availability and sorting controls.

Opening a service shows its full description, freelancer identity, rating, price, delivery time and capacity. The client can make a decision from complete information before adding the service to the cart.

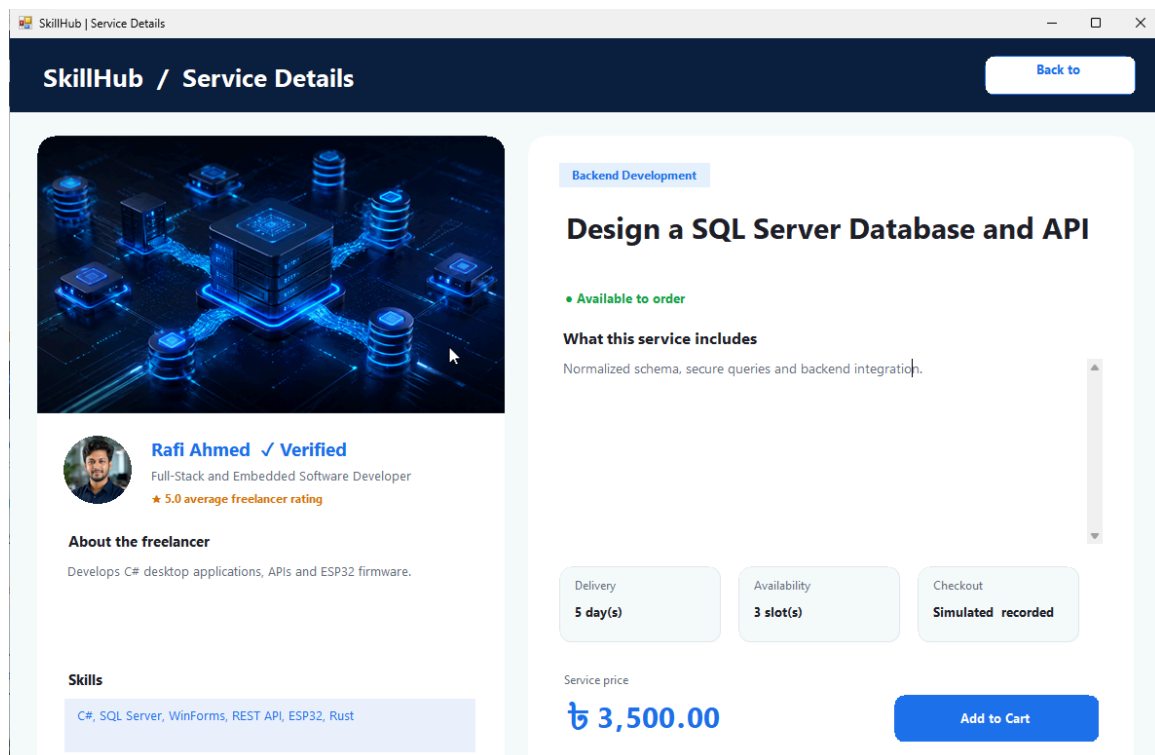


Figure 15. Service Details form with freelancer and purchase information.

Cart, Checkout and Order Record

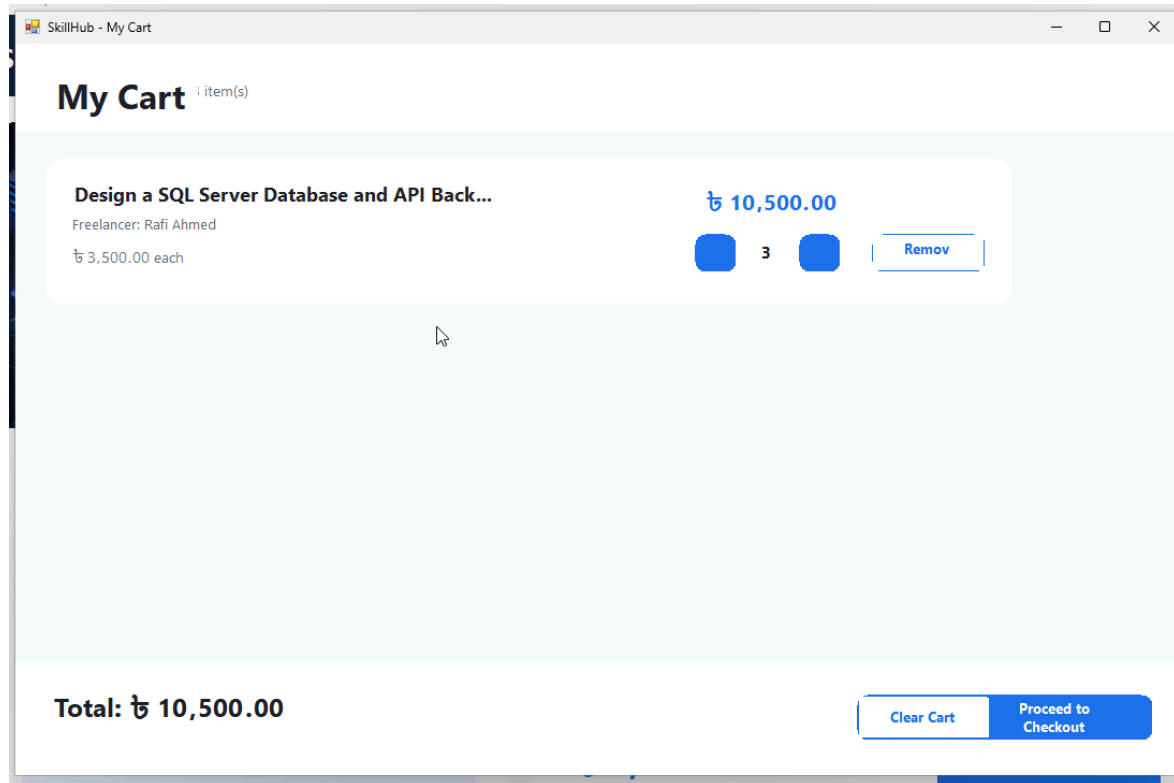


Figure 16. Client cart with quantity, price, subtotal, total and checkout controls.

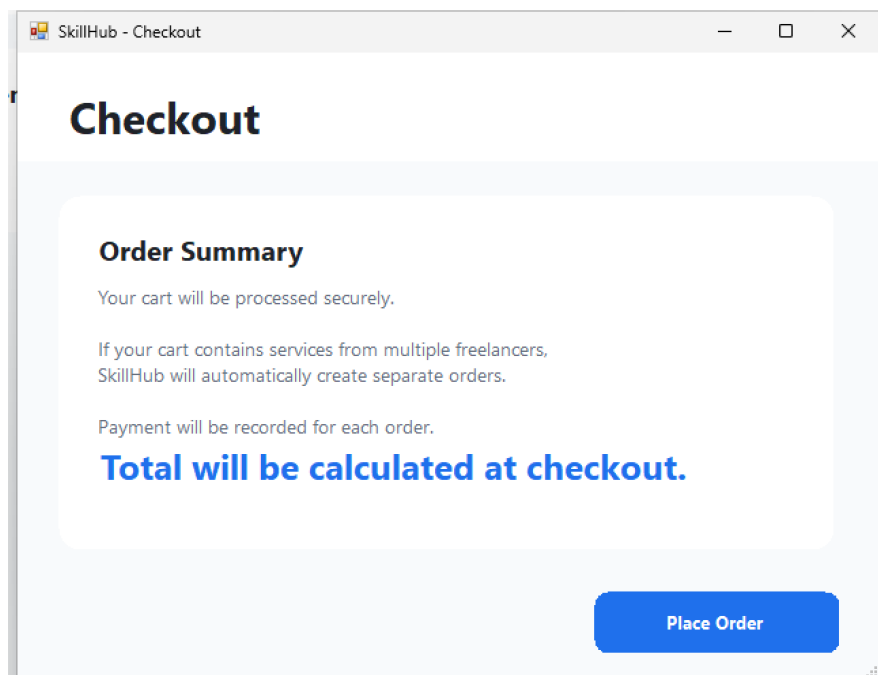


Figure 17. Simulated checkout confirmation and order summary.

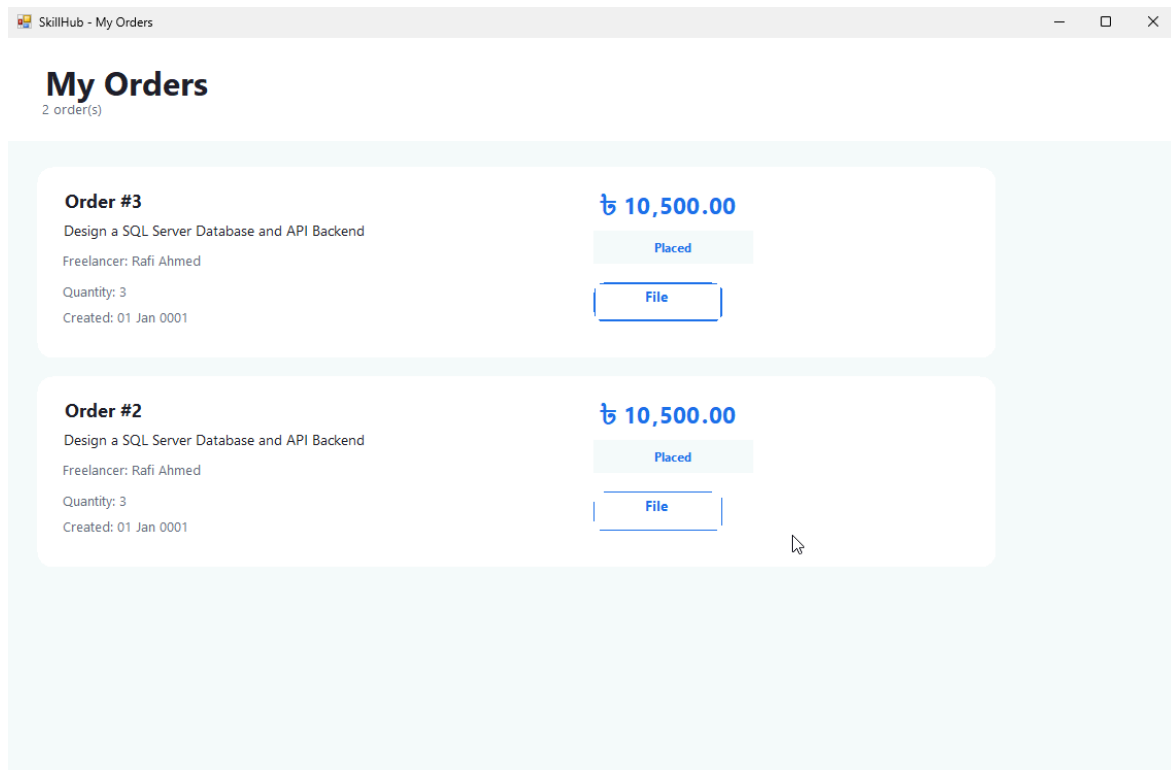


Figure 18. Client order history after checkout.

Freelancer Module

A freelancer maintains a professional profile and the services attached to the signed-in FreelancerId. Manage Services combines the selected record, editor fields, service image, capacity and active status with Create, Update, Deactivate and Clear actions.

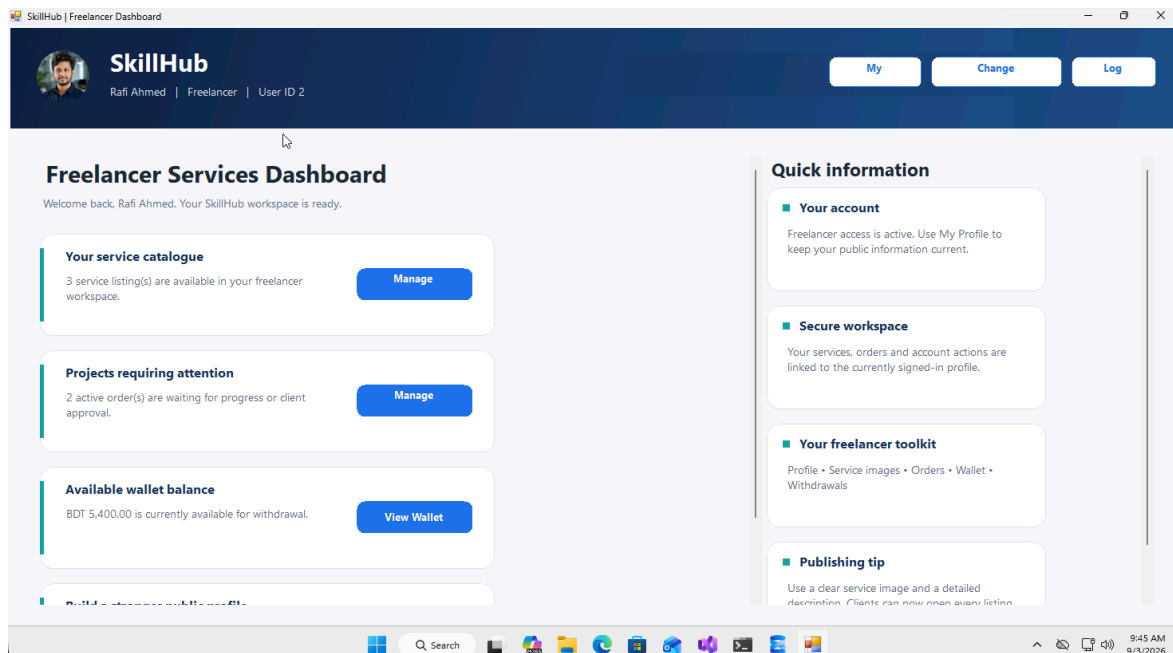


Figure 19. Freelancer profile and dashboard information.

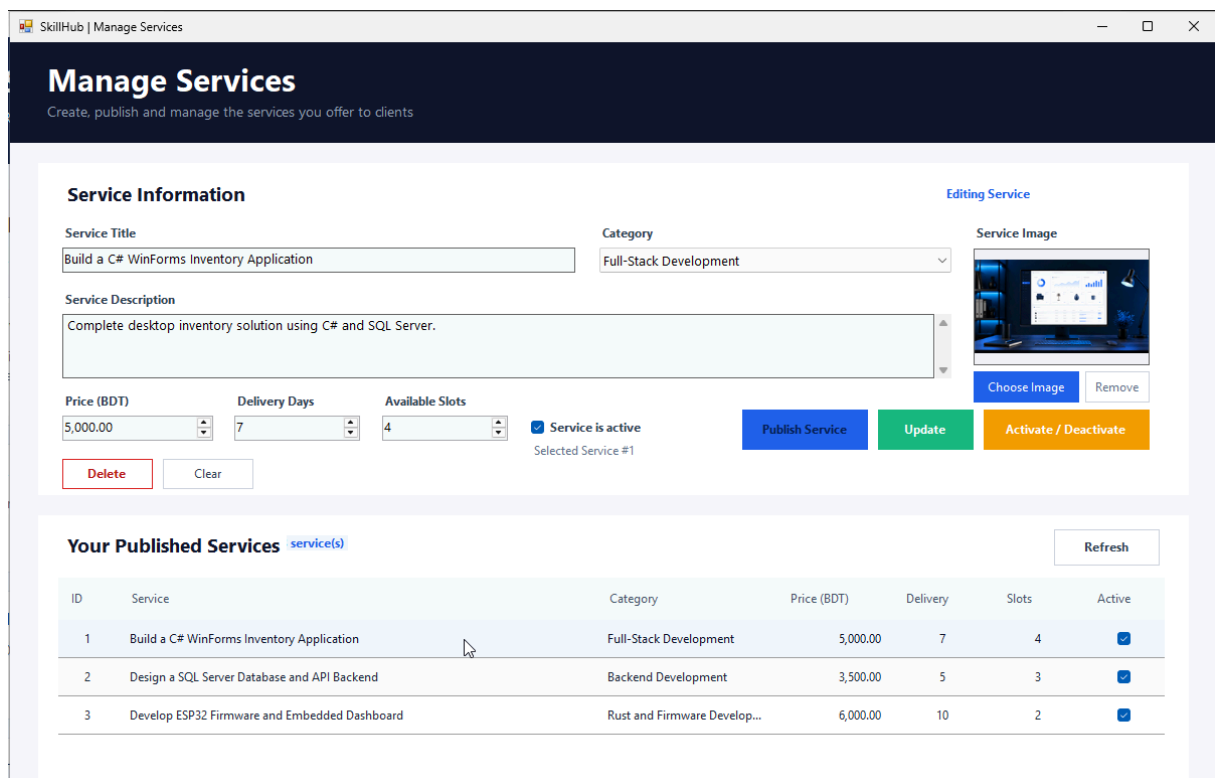


Figure 20. Manage Services form with listing data, image, capacity and CRUD actions.

Capacity and Order Handling

AvailableSlots works as marketplace stock. A service with zero capacity is unavailable to new clients, while an existing assigned order remains visible to the correct freelancer and follows the permitted status sequence.

SkillHub | Manage Services

Manage Services

Create, publish and manage the services you offer to clients

Service Information

Service Title

Category

Android Application Development

Service Description

Price (BDT)

0.00

Delivery Days

1

Available Slots

1

Service is active

☒

No service selected

Delete

Clear

Publish Service

Update

Activate / Deactivate

Create New Service

Service Image

Technology service

New SkillHub service

Choose Image

Remove

Your Published Services

service(s)

Refresh

ID	Service	Category	Price (BDT)	Delivery	Slots	Active
1	Build a C# WinForms Inventory Application	Full-Stack Development	5,000.00	7	0	☒
2	Design a SQL Server Database and API Backend	Backend Development	3,500.00	5	3	☒
3	Develop ESP32 Firmware and Embedded Dashboard	Rust and Firmware Develop...	6,000.00	10	2	☒

Figure 21. Freelancer capacity setting in Manage Services.

SkillHub | Explore Services

SkillHub

Back
View Cart

Find the right expert for your project

Search services, compare freelancers and open any card for full details.

Search

C# win

Category

All categories

Sort by

Recommended

Available now

☒

Explore services

0 service(s) found

No services match these filters

Try a different keyword, category or sorting option.

Figure 22. The same service shown as unavailable in the client marketplace.

17

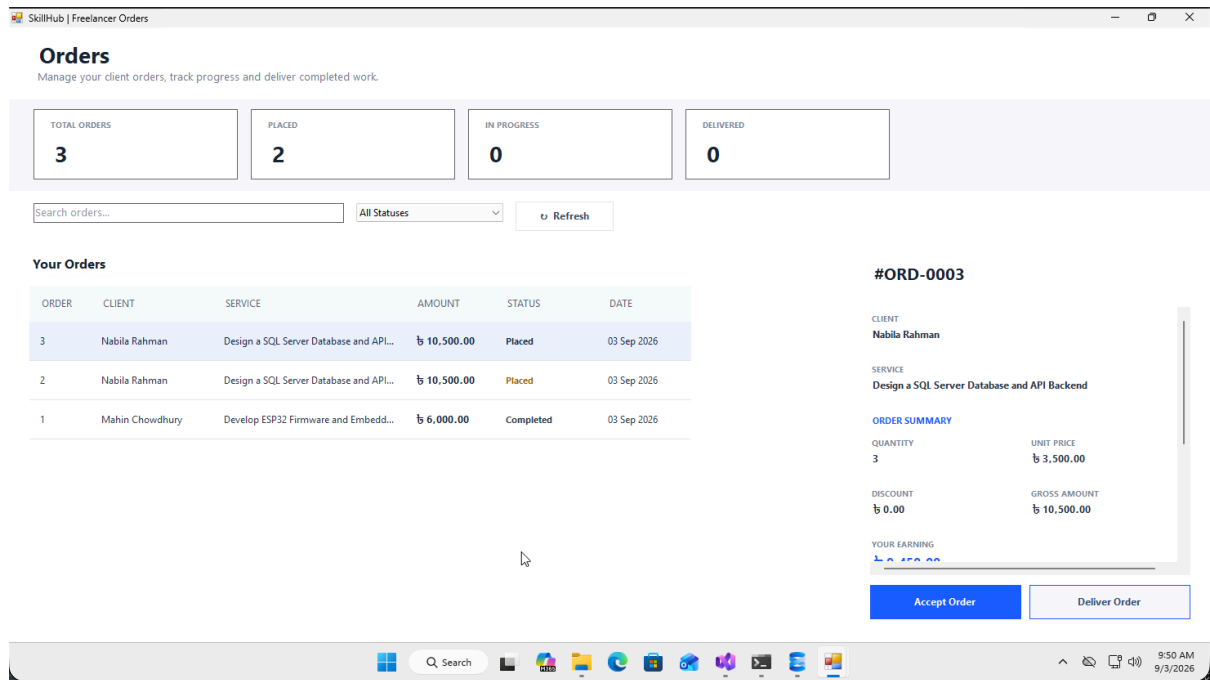


Figure 23. Freelancer order-management screen with role-owned order data and status action.

Delivery Approval and Earnings

The client approves a delivered order. Completion releases the configured freelancer amount, and the corresponding wallet screen shows the resulting credit and balance.

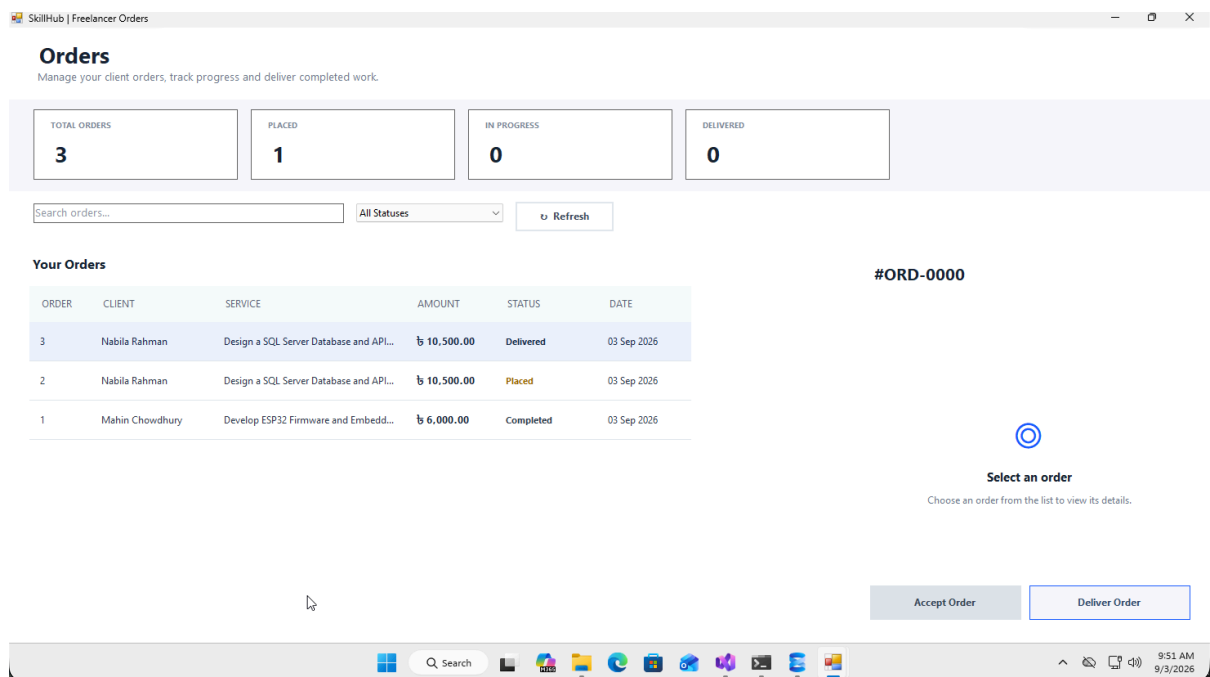


Figure 24. Delivered order waiting for client approval.

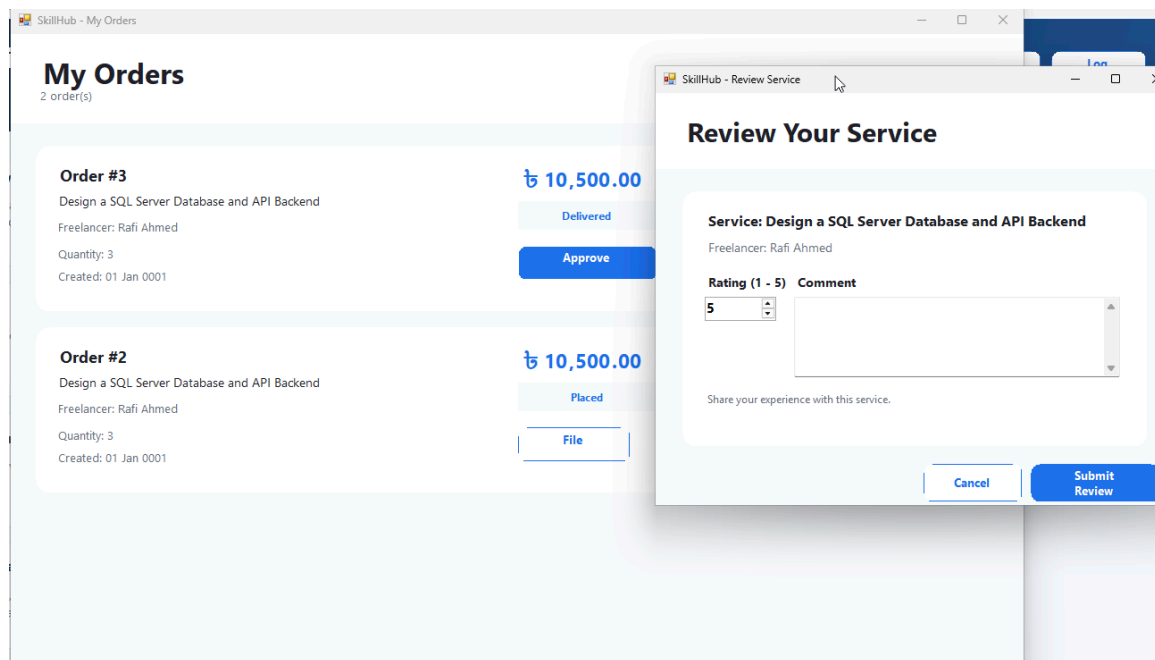


Figure 25. Client review and completion action for the delivered service.

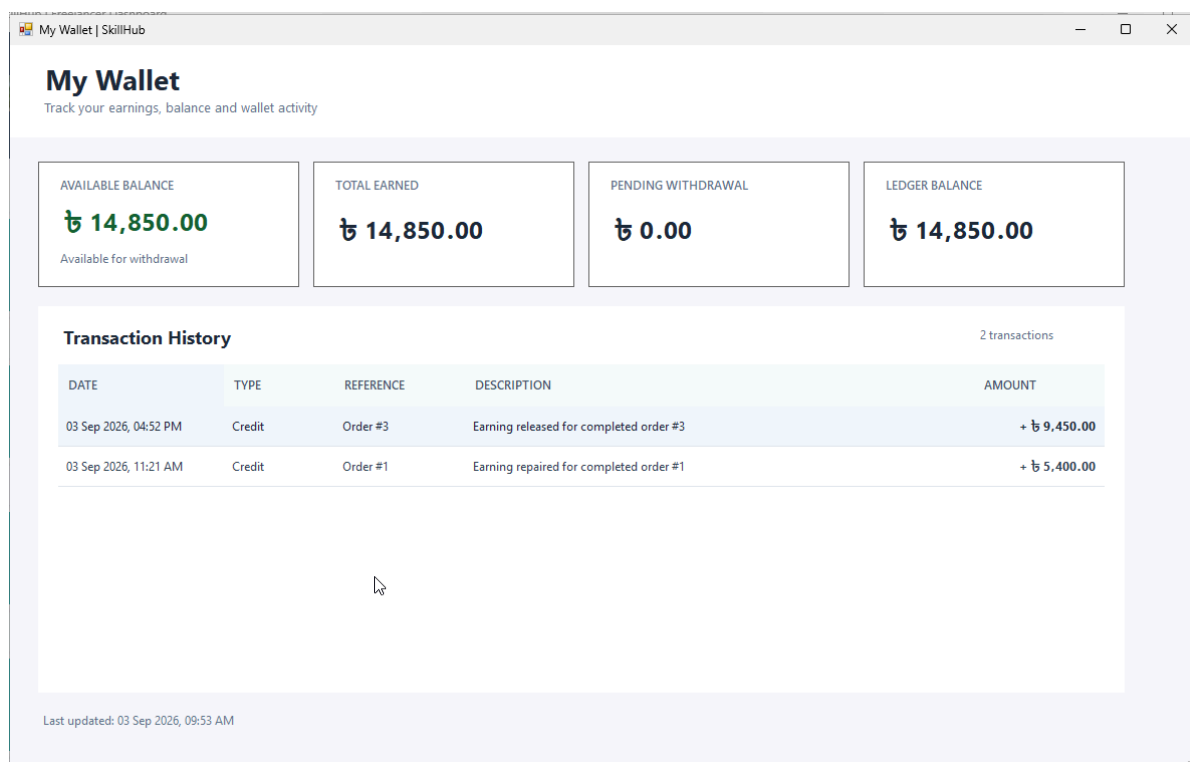


Figure 26. Freelancer wallet after the completed-order credit.

Super Admin Module

The Super Admin dashboard gives one entry point to platform governance. Administrative forms use the same database records created by clients and freelancers, so reports and decisions remain connected to the actual transaction history.

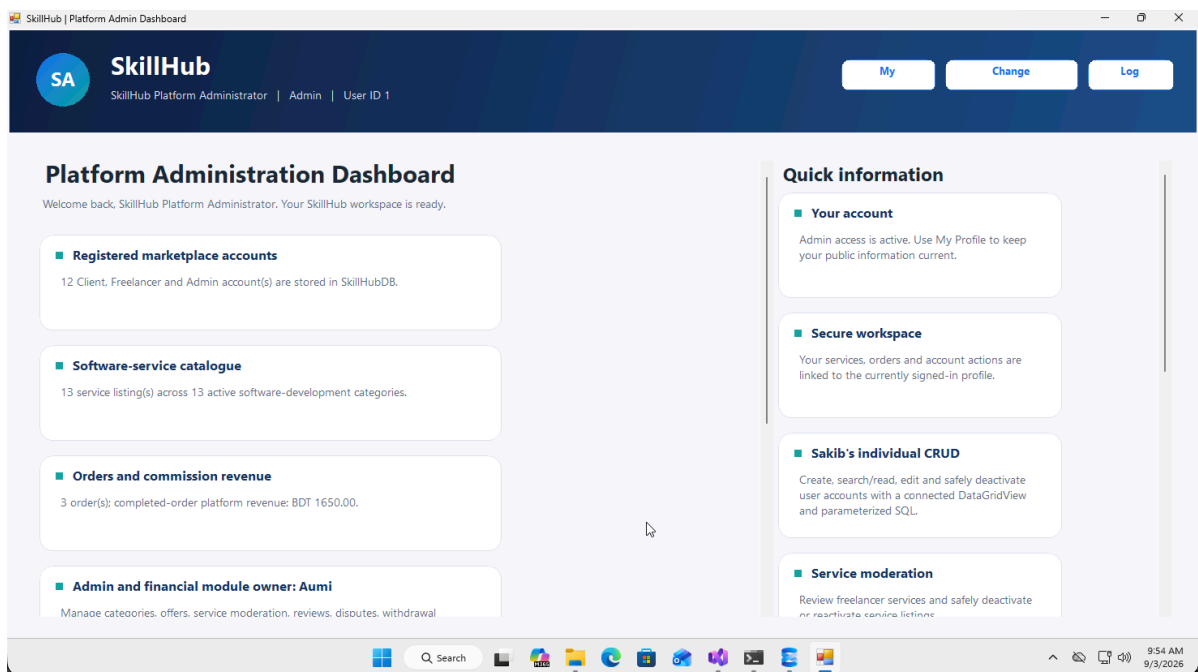


Figure 27. Platform Administration Dashboard.

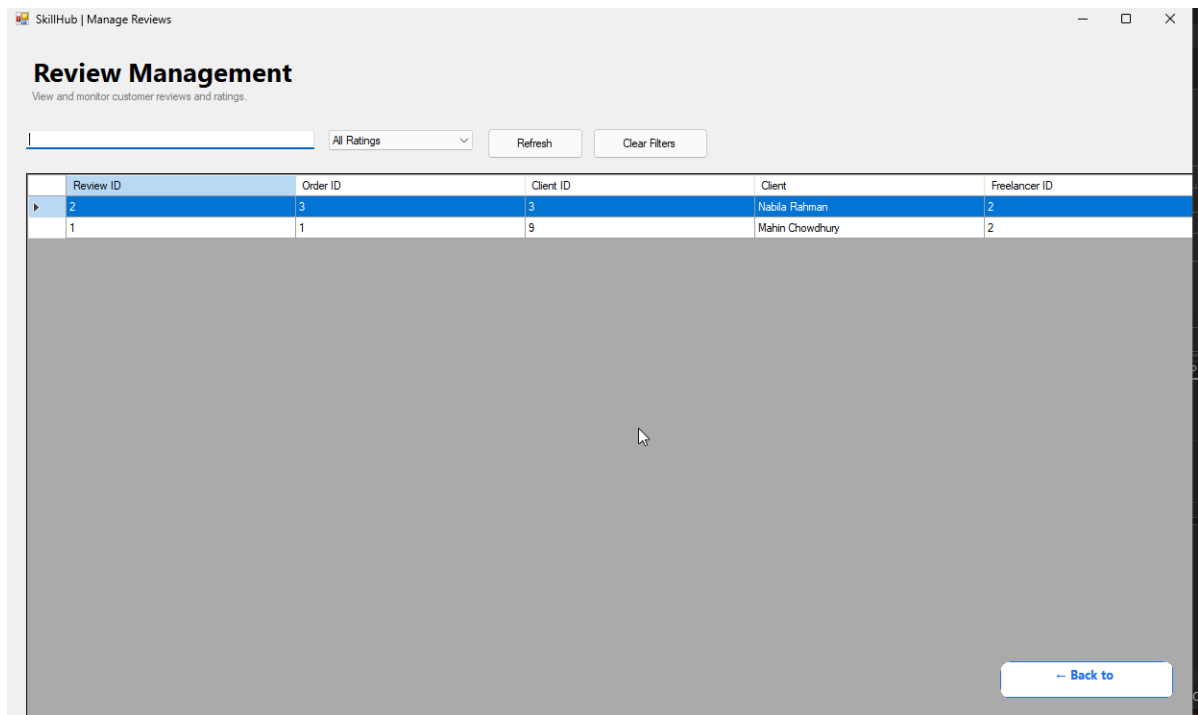


Figure 28. Review Management screen for moderation and verification.

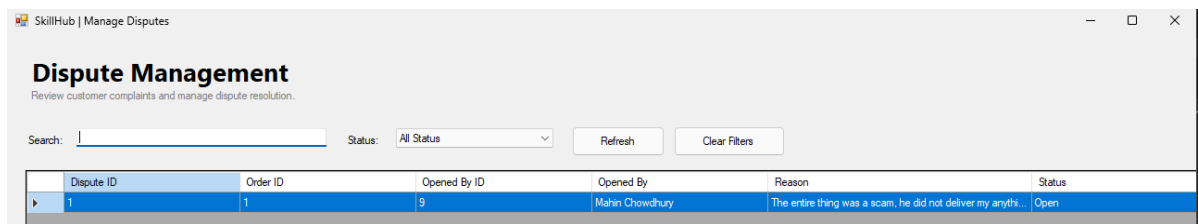


Figure 29. Dispute Management screen with the selected complaint record.

Integration, Validation and Verification

The project was integrated into one solution, one Program.Main entry point, one App.config connection definition and one SkillHubDB database. Shared models, services, repositories and theme rules allow the role modules to operate as one application.

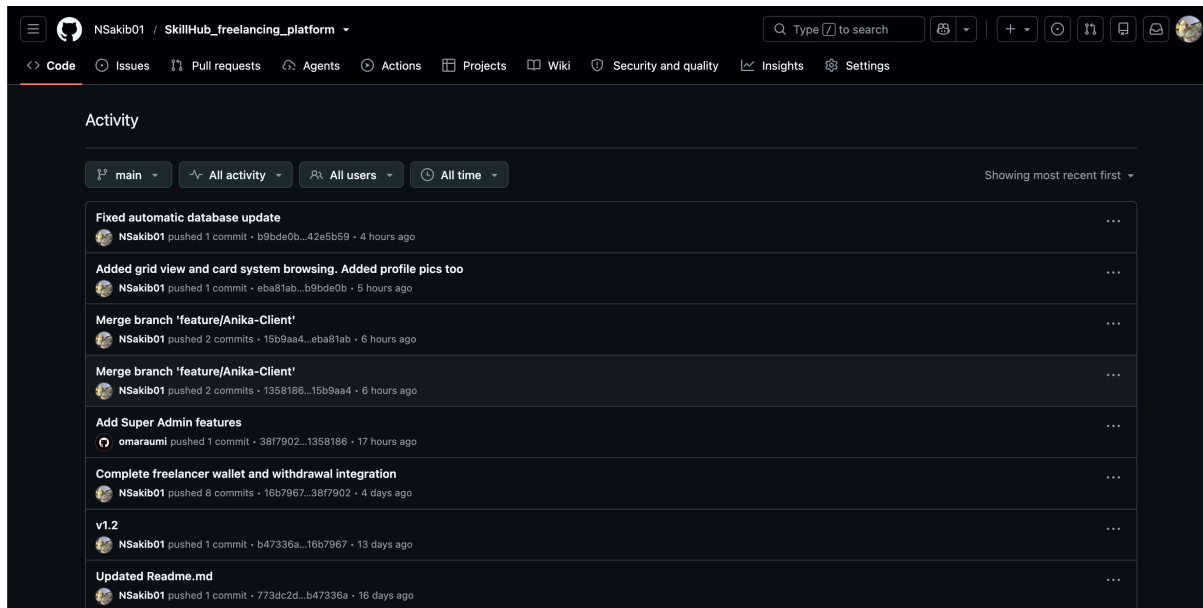


Figure 30. GitHub contribution evidence for the database-foundation branch.

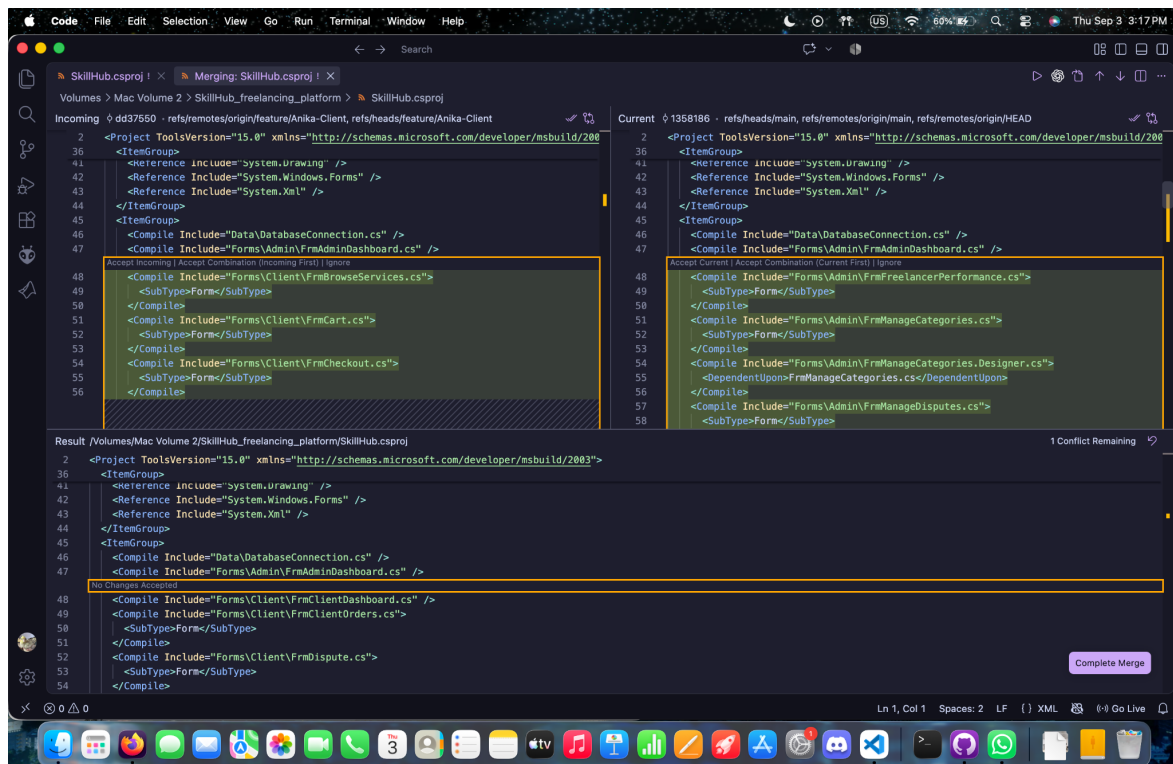


Figure 31. Integration conflict evidence resolved during final project assembly.

8. Conclusion

SkillHub was completed as an integrated three-role freelance marketplace for software services. The final application connects secure authentication, session-based authorization, normalized SQL Server data, account and service CRUD, a visual client marketplace, cart and checkout, order progress, delivery approval, ratings, disputes, wallet credits, withdrawals and administrative reports.

The project satisfies the required OOP2 focus by combining a C# graphical desktop interface with structured classes, reusable services, repository-based data access and one shared database. Validation occurs at the interface, service and database levels, while parameterized commands, account-status checks, password hashing and role-based data isolation improve reliability and security.

The final screenshots and verification results show that the work is no longer a design blueprint. The application builds successfully, the 16-table database is available, each role reaches its own functions, and the main client-to-freelancer transaction completes through payment, delivery approval and earnings. A real payment gateway and production deployment remain outside the academic scope; all financial actions in this version are simulated and stored in the database.